

1

No notes for this slide

qaware.de

UI/UX Qualitätssicherung
Frontends bauen, die halten

Charlie Harding
charlie.harding@qaware.de

QAWARE
SOFTWARE CONSULTING

2

Kurz zu mir: Ich bin Charlie.

Wer bin ich?

Charlie Harding

- 2016–2020 B.Eng. + M.Eng. Computer Science, University of Bristol
- 2018–2019 Erasmus-Student an der TU München
- seit 2020 QAware GmbH München (Senior Software Engineer)
 - Schwerpunkt Frontend-Entwicklung

#gernPerDu



QAWARE
SOFTWARE CONSULTING

- 1 Ich habe in Bristol, England, Informatik studiert,
- 2 mit einem Jahr in München als Erasmus-Student.
- 3 Seit 2020 arbeite ich bei der QAware in München als Senior Software Engineer
- 4 mit Fokus auf Frontend-Entwicklung.

Warum erzähle ich das? Weil ich viele der typischen Frontend-Probleme selbst in Projekten gesehen habe: inkonsistente UI, schwer wartbarer Code, und Accessibility, die zu spät mitgedacht wird.

3

Kurzer Überblick: Wir starten mit der Frage, warum Frontend-Qualität überhaupt ein Thema ist.

Agenda

- 01 Motivation – Warum ist Frontend-Qualität so wichtig?
- 02 TypeScript – Typen als Sicherheitsnetz und Dokumentation
- 03 Linting & Formatting – Automatisierte Standards und CI
- 04 Architektur – Komponentenqualität und Struktur
- 05 Accessibility – Qualität aus Nutzerperspektive
- 06 Design-Systeme – Konsistenz und wiederverwendbare Patterns
- 07 Learnings / Q&A
- 08 Übung

QAWARE
SOFTWARE CONSULTING

Dann schauen wir uns TypeScript, Linting und Architektur als konkrete Werkzeuge an.

Danach geht es um Accessibility und Design-Systeme — also Qualität aus Nutzersicht.

Zum Schluss: Learnings und eure Fragen. Aber ihr könnt gerne jederzeit Fragen stellen — nicht erst am Ende.

Dann kommen wir zur Übung, wo ihr das Gelernte direkt anwenden könnt.

4

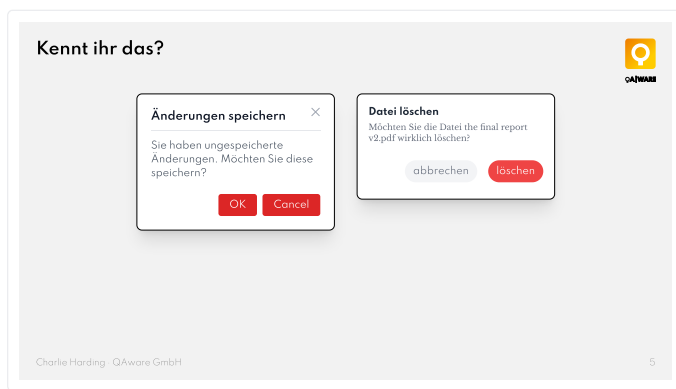
Dann würde ich schon anfangen, und zwar mit meiner Motivation:

01 Motivation
Warum ist Frontend-Qualität so wichtig?

QAWARE
SOFTWARE CONSULTING

Warum beschäftigen wir uns überhaupt damit, wie gut das Frontend gebaut ist?

5



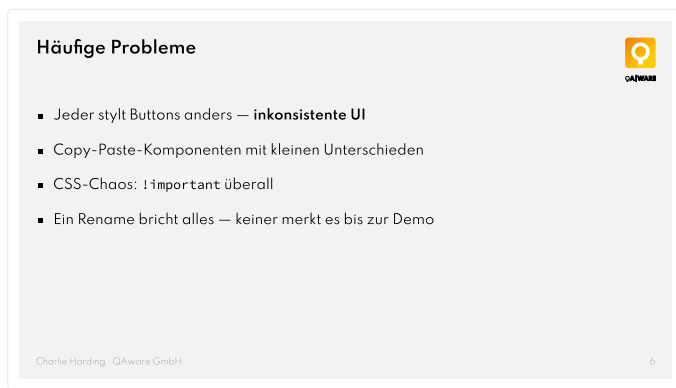
Kurze Frage an euch: Wer hat schon mal in einem Teamprojekt gearbeitet, wo das Frontend irgendwann chaotisch wurde?

1 Hier ein Beispiel: Jemand baut einen Dialog zum Speichern.

2 Dann baut jemand anderes einen zweiten Dialog — diesmal zum Löschen. Sieht schon anders aus.

3 Später wird die Button-Farbe im Lösch-Dialog auf Rot geändert — und die Änderung hat unerwartete Auswirkungen auf den ersten Dialog, was erst viel später auffällt.

6



1 Jeder stylt anders — Buttons sehen auf jeder Seite anders aus.

2 Statt Komponenten wiederzuverwenden, wird copy-pasted und leicht angepasst.

3 CSS wird mit !important überschrieben, weil niemand mehr durchblickt.

4 Und wenn jemand etwas umbenennt, fällt es erst in der Demo auf. Einer dieser Buttons funktioniert nicht, weil das Event umbenannt wurde.

7



Das sind die Symptome. Aber wie eskaliert das eigentlich über die Zeit?

1 Woche 1: Alles läuft. Jeder baut sein Feature, man kommt schnell voran. Qualität? Kommt später. Erstmal soll es funktionieren.

2 Woche 4: Jemand merkt, dass sein Feature bei einem Teamkollegen anders aussieht. Die Abstände sind anders, die Farben stimmen nicht überein. Man fängt an, Dinge mit !important zu überschreiben. Es funktioniert — irgendwie.

3 Woche 8: Jetzt traut sich keiner mehr, etwas anzufassen. Jede Änderung hat unerwartete Nebenwirkungen. Refactoring? Zu riskant. Neue Features? Dauern doppelt so lang. Das Projekt ist technisch am Limit — und es ist erst Woche 8.

8

Frontend-Qualität ≠ nur UI



 Korrektheit Tut es, was es soll?	 Konsistenz Sieht es überall gleich aus? Funktioniert es überall gleich?
 Wartbarkeit Kann jemand anderes es ändern?	 Nutzbarkeit Können alle es bedienen?

Charlie Harding · QAware GmbH 8

Frontend-Qualität ist mehr als nur hübsch aussehen.
Es gibt vier Dimensionen:

- 1 Korrektheit — macht das Feature, was es soll?
Werden Daten richtig angezeigt?
- 2 Konsistenz — sieht es überall gleich aus? Oder hat jede Seite eigene Styles?
- 3 Und nicht nur optisch: funktioniert es auch überall gleich? Gleiche Interaktionen, gleiches Verhalten.
Wenn ein Button einen Bestätigungsdialog zeigt — tun das alle ähnlichen Buttons auch?
- 4 Wartbarkeit — kann ein Teamkollege euren Code verstehen und ändern, ohne etwas kaputt zu machen?
- 5 Und Nutzbarkeit — können alle Nutzer die Anwendung bedienen? Auch mit Tastatur, Screenreader, oder eingeschränktem Kontrast?

9



02 TypeScript
Typen als Sicherheitsnetz und Dokumentation

In diesem Kapitel schauen wir uns TypeScript an — warum es existiert, was es uns bringt, und wie es konkret in der Praxis hilft.

10

JavaScript-Quiz: Arrays



JavaScript ist die Sprache des Webs — aber sie hat ihre Eigenheiten.

Was gibt das aus?

```
[1, 2, 3] + [4, 5, 6]; // → "1,2,34,5,6"
```

Charlie Harding · QAware GmbH 10

JavaScript ist die Sprache des Webs. Jeder Browser kann es, jedes Frontend benutzt es. Aber JavaScript hat Eigenheiten.

Die Sprache wurde initial so gebaut, damit Fehler möglichst selten auftauchen — auch wenn ein Fehler eigentlich besser als eine stille Fehlfunktion wäre.

- 1 Kleines Quiz — was ergibt Array plus Array?
- 2 Ein String! "1,2,34,5,6". Arrays werden zu Strings konvertiert und konkateniert.

11

JavaScript-Quiz: Booleans



Was gibt das aus?

```
true + true + true; // → 3
```

Charlie Harding · QAware GmbH

11

true + true + true?



3. Booleans werden zu Zahlen: true ist 1.

12

JavaScript-Quiz: Argumente



Was gibt das aus?

```
function add(a, b) {  
  return a + b;  
}  
add(1, 2, 3); // → 3  
  
add(1); // → NaN
```

Charlie Harding · QAware GmbH

12

add mit drei Argumenten, obwohl die Funktion nur zwei erwartet?



3. Das dritte Argument wird einfach ignoriert. Kein Fehler.



Und jetzt eine Bonusfrage: Was passiert, wenn wir nur ein Argument übergeben?



b wird auf **undefined** gesetzt, und 1 + undefined ergibt NaN (not a number). Wieder kein Fehler, aber ein unerwartetes Ergebnis.

13

JavaScript-Quiz: String-Arithmetik



Was gibt das aus?

```
'5' + 3; // → "53"  
'5' - 3; // → 2
```

Charlie Harding · QAware GmbH

13

String plus Zahl?



Konkatenation: "53". Und String minus Zahl?



Und jetzt die Frage: Was macht derselbe String mit Minus?



Plötzlich Arithmetik: 2. JavaScript entscheidet je nach Operator.

14

JavaScript-Quiz: Tippfehler



Was gibt das aus?

```
const user = {  
  name: 'Alice',  
  delete() {  
    server.deleteUser(this.name);  
  },  
};  
user.naem; // → undefined  
user.deelte(); // → ✖ TypeError: user.deelte is not a function
```

Charlie Harding · QAware GmbH

14

user.naem — ein Tippfehler bei einer Property.



undefined. Kein Fehler. Und user.deelte() — ein Tippfehler bei einer Methode?



Jetzt endlich ein Fehler — aber erst zur Laufzeit. TypeError.

All diese Bugs würde TypeScript sofort erkennen — beim Tippen, nicht erst zur Laufzeit.

15

Warum TypeScript?



JavaScript + Typen = TypeScript

```

interface Item { name: string; price: number }

function calculatePrice(items: Item[], taxRate: number): number {
  let total = 0;
  items.forEach(item => { total += item.price; });
  return total * (1 + taxRate);
}

calculatePrice([{ name: 'Book', price: 10 }], '19%');

```

Charlie Harding · QAware GmbH
15

Jetzt haben wir gesehen, dass JavaScript für sich allein durchaus riskant sein kann. Genau da kommt TypeScript ins Spiel: Es ergänzt die ursprünglich ohne Compiler gebaute Sprache um einen Compiler.

1 TypeScript ist im Kern einfach: JavaScript mit Typen.

2 Hier ist dann ein Beispiel: Wir haben eine Funktion, die den Gesamtpreis berechnet. Sie erwartet ein Array von Items und einen Steuersatz. Wer kann mir sagen, was hier das Ergebnis ist?

3 Der Trick ist, dass wir `taxRate` als String übergeben, obwohl die Funktion eine Zahl erwartet. Das Ergebnis ist NaN — aber das passiert erst zur Laufzeit.

4 Mit TypeScript machen wir die Schnittstelle explizit: `items` ist ein Array von `Item`, und `taxRate` ist eine Zahl.

5 Wir können auch angeben, dass die Funktion eine Zahl zurückgibt. Das hilft, wenn man den Rückgabewert später benutzt.

16

Warum TypeScript?



JavaScript + Typen = TypeScript

```

interface Item { name: string; price: number }

function calculatePrice(items: Item[], taxRate: number): number {
  let total = 0;
  items.forEach(item => { total += item.price; });
  return total * (1 + taxRate);
}

calculatePrice([{ name: 'Book', price: 10 }], '19%');
Argument of type 'string' is not assignable to parameter of type 'number'.

```

Charlie Harding · QAware GmbH
16

Wenn wir dann `taxRate` als String übergeben, meldet der Compiler den Fehler sofort.

Typen als Dokumentation



- Signatur in der IDE

```
function calculatePrice(items: Item[], taxRate: number): number
```

- Parametertypen sind dokumentiert: items: Item[], taxRate: number
- Rückgabewert ist klar: : number
- Falsche Aufrufe fallen sofort auf: calculatePrice(..., '19%')
- Kein Raten, kein Strg+F
- Fehler beim Schreiben oder Build

Charlie Harding - QAware GmbH

17

TypeScript macht den Code nicht nur sicherer, sondern auch lesbarer.

- 1 In der IDE (bspw. VS Code oder IntelliJ) sieht man die Signatur der Methode als Tooltip.
- 2 Die Typen der Parameter sind dokumentiert.
- 3 Der Rückgabebetyp sagt sofort, was zurückkommt.
- 4 Wenn wir dann **taxRate** als String übergeben, meldet der Compiler den Fehler sofort.
- 5 Das spart Zeit im Alltag: weniger Suche, weniger Missverständnisse.
- 6 Das ist der Kernvorteil: Fehler werden früh abgefangen. Nicht zur Laufzeit, sondern bereits beim Schreiben oder beim Build.

Bessere Komponenten-APIs



```
@Component({ ... })
class PriceBadgeComponent {
  readonly amount = input.required<number>();
  readonly currency = input<'EUR' | 'USD'>('EUR');
}

<app-price-badge [amount]="19" currency="GBP" />
                  ^^^
Type '"GBP"' is not assignable to type '"EUR" | "USD"'
```

Charlie Harding - QAware GmbH

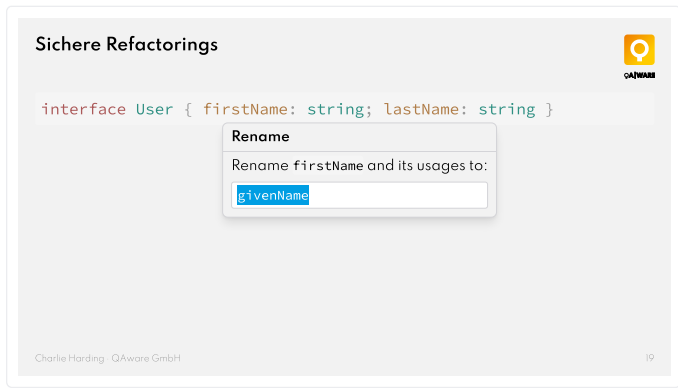
18

TypeScript hilft nicht nur bei Funktionen, sondern auch bei Komponenten, bspw. in React, Vue oder Angular. Inputs bilden die öffentliche Schnittstelle.

- 1 Hier haben wir eine Angular-Komponente mit zwei Inputs: **amount** ist eine required number, und **currency** ist ein optionaler String, der nur "EUR" oder "USD" sein darf.
- 2 Wenn ein Input als required markiert ist, muss er auch übergeben werden. Ansonsten gibt es einen Fehler.
- 3 Wenn der Typ nicht stimmt, gibt es ebenfalls einen Fehler. Hier wird eine Zahl erwartet, aber ein String übergeben.
- 4 Mit einer Zahl funktioniert es.
- 5 Und wenn wir einen ungültigen Wert für **currency** übergeben, gibt es auch einen Fehler. Das ist besonders hilfreich bei Enums oder Union Types.

19

Sichere Refactorings



```
interface User { firstName: string; lastName: string }
```

Rename
Rename firstName and its usages to:
givenName

Charlie Harding · QAware GmbH 19

Mit Typannotationen können wir leicht finden, welche Objekte ein bestimmtes Interface verwenden.

- 1 In IDEs geht das automatisch, zum Beispiel mit „Show Usages“.
- 2 Da sieht man alle Stellen, an denen das Interface verwendet wird.
- 3 Das kann dann für Refactorings genutzt werden:
- 4 Mit dem Rename-Tool der IDE aktualisieren wir alle Verwendungen, ohne Felder anderer Objekte zu verändern, die zufällig gleich heißen.

20

Sichere Refactorings



```
interface User { givenName: string; lastName: string }

function mapUser(user: RawUser): User {
  return {
    firstName: user.vorname,
    lastName: user.nachname,
  };
}
```

Object literal may only specify known properties, and 'firstName' does not exist in type 'User'.

Charlie Harding · QAware GmbH 20

Und wenn am Ende doch etwas übersehen wurde, warnt TypeScript vor unbekannten Properties.

21

API-Verträge

- Shared Types
- Generiert aus API-Verträgen (z. B. OpenAPI- oder GraphQL-Schema)
- Backend ändert Response → Frontend-Build bricht sofort
- TypeScript macht Änderungen früh sichtbar — bevor sie in Produktion gehen

Charlie Harding · QAware GmbH 21

TypeScript ist nicht nur für lokalen Code nützlich — es kann auch API-Grenzen absichern.

- 1 Shared Types bedeutet: Backend und Frontend sprechen dieselbe Sprache.
- 2 Viele Teams generieren diese Typen automatisch aus API-Verträgen. OpenAPI und GraphQL-Schemas sind dabei maschinenlesbare Beschreibungen der Schnittstelle: Welche Endpunkte es gibt, welche Felder kommen und welche Typen sie haben. Diese lassen sich wiederum aus dem Backend-Code generieren.
- 3 Wenn das Backend eine Property umbenennt oder entfernt, schlägt der Frontend-Build sofort an — kein manuelles Suchen.
- 4 Damit schließen wir das TypeScript-Kapitel: Typen machen den Code sicherer und lesbarer und helfen, Änderungen früh sichtbar zu machen — lange bevor sie in Produktion gehen.

22



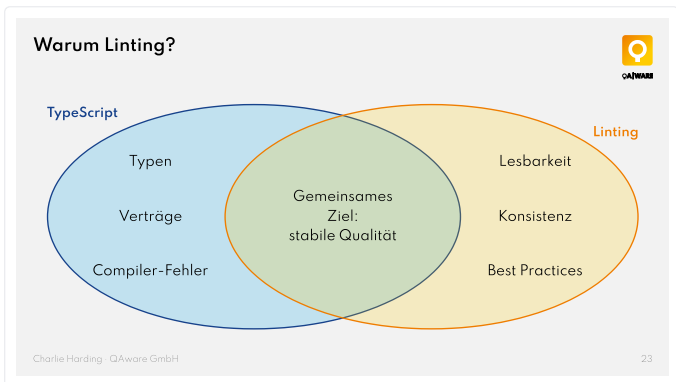
Wir haben gerade gesehen: TypeScript gibt uns ein starkes Sicherheitsnetz für Typen.

Der nächste Schritt ist, auch die Codequalität selbst abzusichern.

Genau da helfen Linting und Formatting: weniger Inkonsistenzen, klarere Standards und ruhigerer Entwicklungsalltag.

Wir schauen uns das jetzt Schritt für Schritt an, erst im Editor und danach im CI-Workflow.

23

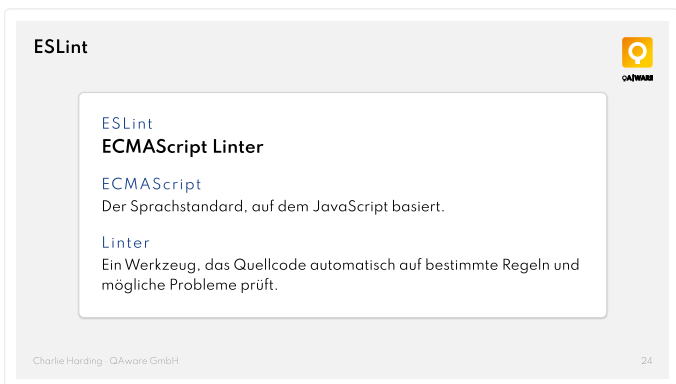


TypeScript war Schritt 1: Typen und Verträge absichern.

1 Der nächste Schritt ist Linting: Lesbarkeit, Konsistenz und Best Practices.

2 Zusammen ergibt das: frühes Feedback und stabilere Qualität im Alltag.

24



ESLint steht für ECMAScript Linter. ECMAScript ist der Sprachstandard, auf dem JavaScript basiert.

Ein Linter ist allgemein ein Werkzeug, das Quellcode automatisch auf bestimmte Regeln und mögliche Probleme prüft.

Kurz gesagt: ESLint prüft Code auf typische Probleme und hilft beim Einhalten von Stilregeln.

ESLint ist dabei im JavaScript- und TypeScript-Umfeld der am weitesten verbreitete Linter, aber nicht die einzige Option.

Jetzt schauen wir uns an, wie das direkt im Code aussieht.

25



Wir starten mit einer der wichtigsten Basisregeln: **no-undef**.

Nur definierte Variablen verwenden. Das hilft direkt gegen Tippfehler und vergessene Definitionen.

26

ESLint in Aktion (JavaScript)

1. Nur definierte Variablen verwenden — no-undef
 2. Unbenutzte Variablen vermeiden — no-unused-vars

verboten	erlaubt
<pre>const length = items.length; return items.length;</pre>	<pre>const length = items.length; return length;</pre>

Charlie Harding · QAware GmbH 26

Als Nächstes kommt **no-unused-vars**.

Alles entfernen, was nicht genutzt wird. Das hält den Code kompakter und reduziert Verwirrung beim Lesen.

27

ESLint in Aktion (JavaScript)

1. Nur definierte Variablen verwenden — no-undef
 2. Unbenutzte Variablen vermeiden — no-unused-vars
 3. Bedingungen immer mit Block — curly

verboten	erlaubt
<pre>if (isAdmin) grantAccess();</pre>	<pre>if (isAdmin) { grantAccess(); }</pre>

🔧 automatisch korrigierbar

Charlie Harding · QAware GmbH 27

Zum Schluss **curly**: Kontrollstrukturen immer mit geschweiften Klammern.

Das macht den Kontrollfluss klarer und verhindert Fehler bei späteren Erweiterungen.

🔧 Und das Beste: Viele Regeln können automatisch korrigiert werden. In der IDE kann man das so konfigurieren, dass solche Probleme direkt beim Speichern behoben werden.

28

ESLint in Aktion (TypeScript)

1. Kein explizites any — @typescript-eslint/no-explicit-any

verboten	erlaubt
<pre>function printValue(value: any,) { return `Wert: \${value}`; }</pre>	<pre>function printValue(value: string number,) { return `Wert: \${value}`; }</pre>

Charlie Harding · QAware GmbH 28

Nachdem wir jetzt TypeScript haben, können wir auch Regeln nutzen, die speziell auf die typischen Fallstricke von TypeScript abzielen.

Hier starten wir mit **no-explicit-any**: Wir vermeiden **any**, damit die Typprüfung nicht umgangen wird. **any** bedeutet im Grunde: „TypeScript, vertraue mir einfach“.

Statt **any** nutzen wir konkrete Typen. Wenn das nicht möglich ist, sind **unknown**, **never** oder Generics meist die bessere Wahl.

29

ESLint in Aktion (TypeScript)

1. Kein explizites any — @typescript-eslint/no-explicit-any
 2. Keine floating Promises — @typescript-eslint/no-floating-promises

verboten	erlaubt
<pre>async function saveUser(user: User,) { ... }</pre> <pre>saveUser(user);</pre>	<pre>async function saveUser(user: User,) { ... }</pre> <pre>await saveUser(user);</pre>

Charlie Harding · QAware GmbH 29

Die zweite wichtige Regel ist **no-floating-promises**.

Asynchrone Aufrufe müssen bewusst behandelt werden, sonst gehen Fehler still verloren. Deshalb wird der Aufruf hier korrekt mit **await** abgefangen.

30

ESLint in Aktion (TypeScript)



1. Kein explizites any — @typescript-eslint/no-explicit-any
2. Keine floating Promises — @typescript-eslint/no-floating-promises
3. **Sichere Template-Strings** — @typescript-eslint/restrict-template-expressions

verboten	erlaubt
<pre>const label = `User: \${user}`; // => "User: [object Object]"</pre>	<pre>const label = `User: \${user.name}`; // => "User: Charlie"</pre>

Charlie Harding · QAware GmbH 30

Zum Abschluss: `restrict-template-expressions` .

In Template-Strings sollen nur passende Typen landen, damit keine unerwarteten Ausgaben entstehen.

Wenn man ein Objekt implizit zu String macht, erhält man oft `[object Object]` . Mit der Regel wird stattdessen der konkrete Wert genutzt, hier `user.name` .

31

Prettier



```
function buildLabel(user) {
  if (user.isActive) {
    return user.firstName + ' ' + user.lastName;
  }
  return "inaktiv";
}
```

Charlie Harding · QAware GmbH 31

Bis jetzt ging es vor allem um Linting-Regeln. Für reines Formatieren nehmen viele Teams zusätzlich Prettier.

🔗1 Das ist ein Beispiel ohne einheitliche Formatierung: alles funktioniert, aber es liest sich anstrengend.

32

Prettier



```
function buildLabel(user) {
  if (user.isActive) {
    return user.firstName + ' ' + user.lastName;
  }
  return 'inaktiv';
}
```

- Kein Streit über Formatierung
- Komplette automatisch in der IDE
- Vereinfacht Merges

Charlie Harding · QAware GmbH 32

Und so sieht derselbe Code nach Prettier aus: konsistente Einrückung, Zeilenumbrüche und einheitlicher Stil.

🔗1 Der große Vorteil: Stilfragen werden automatisch entschieden, damit wir im Team mehr über Logik und weniger über Formatierung diskutieren.

🔗2 Man muss gar nicht darüber nachdenken, weil der Code direkt in der IDE korrigiert wird, sobald man speichert.

🔗3 Außerdem werden Merges einfacher, weil es keine inkompatiblen Formatierungs-Änderungen mehr gibt.

CI Pipeline



🔔 c-harding hat einen neuen Pull Request erstellt: **Add user profile page**

📦 2 Commits, 2 Dateien geändert, 52 Zeilen hinzugefügt, 11 Zeilen gelöscht

Build	✓
Test	✓
Format	✓
Lint	✓

Charlie Harding · QAware GmbH

33

Bis jetzt haben wir Linting und Formatting vor allem als Werkzeuge im Editor gesehen.

Aber viel wichtiger ist eigentlich die Pipeline, die bei einem Pull-Request läuft. Hier ist ein Beispiel, was wir alles hier automatisieren können.

Sobald ein PR erstellt wird, startet die Pipeline. Sie prüft zuerst, dass der Code gebaut werden kann: Im Frontend bedeutet das, dass die TypeScript-Typen geprüft werden.

➤ 1 Dann laufen die Unit-Tests, damit wir sicher sein können, dass die Logik funktioniert.

➤ 2 Wir prüfen die Formatierung. Wenn die IDE richtig konfiguriert ist, kann das hier nie scheitern.

➤ 3 Und dann kommt der Linting-Check.

➤ 4 In diesem Fall gibt es einen Verstoß gegen die Regeln, also bekommen wir eine Warnung. Bevor wir den PR mergen dürfen, muss das Problem behoben werden.

➤ 5 Besser so: Ein Commit später ist alles grün.

Fazit

- Linting und Formatting geben frühes, automatisches Feedback
- TypeScript, ESLint und Prettier ergänzen sich
- Lokal und CI sollten dieselben Regeln erzwingen
- Weniger Stil-Diskussionen, mehr Fokus auf Logik



Charlie Harding · QAware GmbH

34

Das war's dann für automatische Checks. Zusammenfassend:

➤ 1 Linting und Formatting sind das zweite Sicherheitsnetz nach TypeScript.

➤ 2 Die Werkzeuge ergänzen sich: TypeScript für Typen, ESLint für Regeln, Prettier für einheitliches Format.

➤ 3 Wichtig ist die Konsistenz zwischen lokalem Workflow und CI.

➤ 4 Dann haben wir weniger Diskussionen über Stil und mehr Zeit für die eigentliche Fachlogik.

35



Bis jetzt haben wir uns um die Qualität einzelner Codezeilen gekümmert: korrekte Typen, saubere Regeln, einheitliches Format.

Aber selbst wenn jede Zeile für sich gut ist, kann das große Ganze trotzdem unübersichtlich werden. Komponenten, die zu viel wissen. Abhängigkeiten, die sich quer durch die Anwendung ziehen. Logik, die man nicht mehr findet, weil sie irgendwo tief versteckt ist.

Das Ziel dahinter ist immer dasselbe: Änderungen lokal halten und Komplexität begrenzen.

Darum geht es jetzt: Wie schneiden wir Komponenten sinnvoll? Wie halten wir Datenflüsse nachvollziehbar? Und wie legen wir eine Projektstruktur an, die auch noch in einem Jahr verständlich ist?

Kurz gesagt: Qualität auf der Ebene der Struktur.

36



Das Grundprinzip ist einfach: Es gibt zwei Arten von Komponenten.

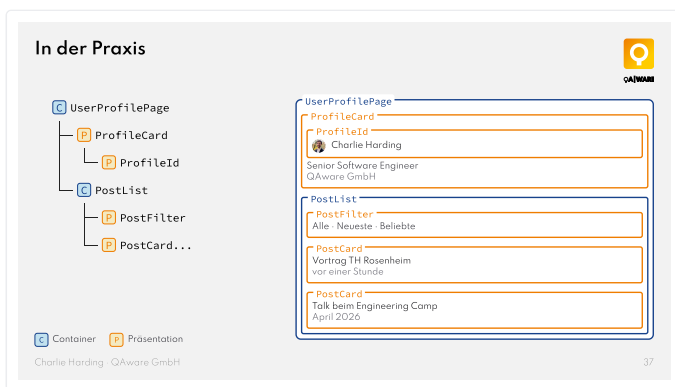
Die Container-Komponente kennt die Daten. Sie lädt sie aus einem Service oder Store, hält den State und koordiniert, was passiert.

Die Präsentationskomponente weiß davon nichts. Sie bekommt ihre Daten über Props, zeigt sie an und schickt Benutzeraktionen als Events nach oben.

Der Datenfluss ist damit klar und vorhersehbar: Props runter, Events rauf. Keine versteckten Abhängigkeiten, kein direkter Zugriff auf globalen State mitten im Template.

1 Eine Komponente, eine klare Verantwortung. Das ist die Grundregel.

2 Dadurch entstehen klareres Verständnis, einfachere Testbarkeit und mehr Flexibilität.



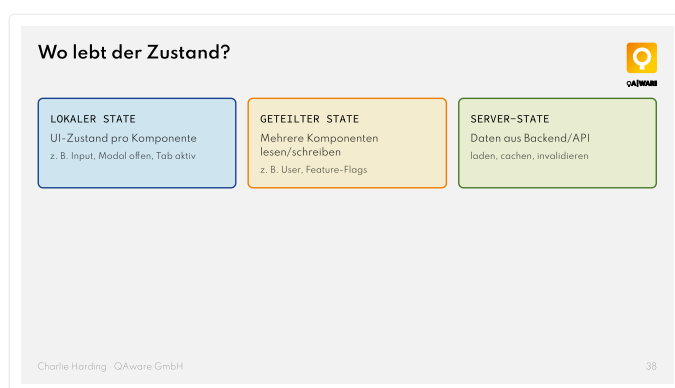
Und so sieht das in einer echten Anwendung aus.

Die **UserProfilePage** ist ein Container — sie koordiniert die gesamte Seite, lädt Nutzerdaten und entscheidet, was gerendert wird.

ProfileCard und ihr Kind **ProfileId** darunter sind reine Präsentationskomponenten: Sie bekommen ihre Daten als Props und zeigen sie an.

Dann gibt es **PostList** — auch ein Container, aber nur für die Beiträge. Sie verwaltet den Filter-State, lädt die Posts und rendert dann **PostFilter** und **PostCard** darunter.

Das Muster wiederholt sich auf jeder Ebene. Die Regel ist immer dieselbe — egal ob die App 5 oder 50 Komponenten hat.



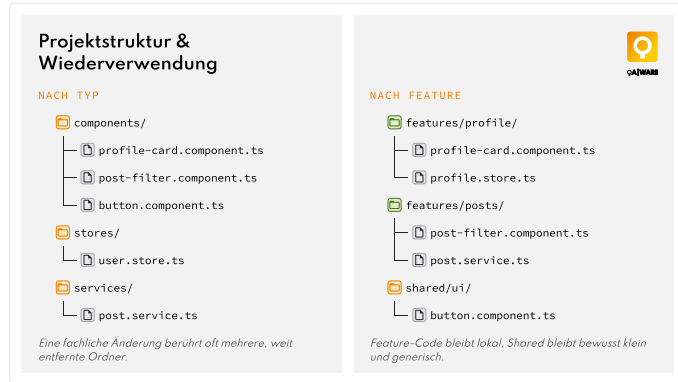
Nach dem Komponentenschnitt kommt die nächste Frage: Wo lebt welcher Zustand?

Erstens lokaler State: Alles, was wirklich nur eine Komponente betrifft. Dazu gehört typischerweise UI-Zustand wie Input-Werte, ob ein Modal offen ist oder welcher Tab aktiv ist.

1 Zweitens geteilter State: Informationen, die mehrere Komponenten brauchen. Den bewusst zentral halten, nicht duplizieren. Dazu gehören zum Beispiel der aktuell eingeloggte User oder globale Feature-Flags.

2 Drittens Server-State: Das sind Backend-Daten, die beispielsweise aus einer Datenbank geladen werden. Die behandeln wir anders als UI-State: laden, cachen, aktualisieren.

Es gibt also keine Einheitslösung für Zustand. Je nach Anwendungsfall und Lebenszyklus wählen wir den passenden Ort.



Hier sehen wir denselben Code in zwei unterschiedlichen Strukturen.

Links nach Dateitypen: Komponenten, Stores, Services in getrennten Ordnern. Das wirkt erstmal ordentlich, aber fachlich zusammengehöriger Code liegt weit auseinander.

1 Eine fachliche Änderung berührt hier oft mehrere, weit entfernte Ordner.

2 Rechts nach Features: Alles, was zu Profile oder Posts gehört, liegt zusammen. Das reduziert Kontextwechsel und hält Änderungen lokaler.

3 Feature-Code bleibt lokal, Shared bleibt bewusst klein und generisch.

Shared ist bewusst klein: nur wirklich generische, wiederverwendbare Bausteine. So bleiben die Grenzen zwischen feature-spezifischem Code und Shared Components klar.

Architektur im Alltag

- **Im Review:** Verantwortungen und Abhängigkeiten sind schneller erkennbar
- **Beim Refactoring:** Änderungen bleiben lokaler und risikoärmer
- **Beim Einstieg:** Neue Leute finden sich im Projekt schneller zurecht
- **Als Grundlage:** Gute Struktur macht Tests, Accessibility und Design-Systeme leichter

Charlie Harding · QAware GmbH

40

Bis jetzt haben wir auf konkrete Architekturentscheidungen geschaut: Komponentenschnitte, State und Projektstruktur.

Und genau daran merkt man im Alltag sehr schnell, ob eine Architektur trägt oder nicht.

1 Das sieht man zuerst im Review. Wenn Verantwortungen klar geschnitten sind, erkennt man schneller, was eine Komponente eigentlich tut und wo mögliche Seiteneffekte liegen.

2 Dann im Refactoring: Gute Struktur hält Änderungen lokaler. Man muss weniger Stellen gleichzeitig anfassen und hat damit weniger Risiko.

3 Und schließlich beim Einstieg ins Projekt: Neue Leute finden sich schneller zurecht, wenn die Struktur die Fachlichkeit widerspiegelt.

4 Deshalb ist Architektur auch die Grundlage für vieles, was danach kommt — Tests, Accessibility und Design-Systeme profitieren alle davon.

Architektur ist also kein Selbstzweck. Gute Struktur reduziert Reibung im Entwicklungsalltag.

41



Bis jetzt ging es um die Qualität im Code. Jetzt geht es darum, wie die Anwendung für alle Menschen funktioniert — mit verschiedenen Fähigkeiten und Kontexten.

42



Ich stelle eine Frage: Wer von euch nutzt Accessibility-Features? Hebt die Hand auf, wenn das auf euch zutrifft.

-  1 Erstens: Screen Reader zum Navigieren im Web.
-  2 Speech-to-Text.
-  3 Zooming auf Websites.
-  4 Untertitel bei Videos.
-  5 Dark Mode.
-  6 Mit einer Hand das Handy bedienen.
-  7 Und das ist der Punkt: Das ist Accessibility. Nicht ein Nischen-Feature für Menschen mit Behinderungen, sondern eure täglichen Situationen. Du hältst einen Kaffee in der Hand und scrollst mit der anderen. Du magst Dark Mode. Du siehst kleinen Text nicht gut und zoomst Websites.

Accessibility ist universell. Deshalb bauen wir es von Anfang an ein.

Was ist Accessibility? 

DAUERHAFT  Sehbehinderung  Motorik  Gehörlos  Kognitiv

SITUATIV  Grelles Licht  Eine Hand belegt  Ton aus  Langsames Internet

Dieselbe Barriere, unterschiedliche Ursachen.

Charlie Harding · QAware GmbH 43

Wir haben gesehen, dass Accessibility uns alle betrifft. Aber was steckt dahinter?

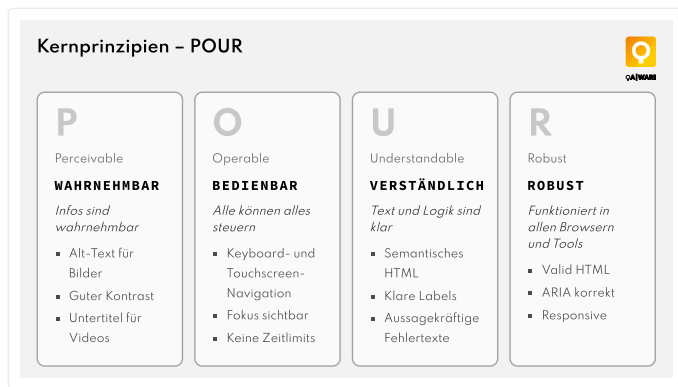
Es gibt zwei Kategorien.

1 Dauerhaft: Menschen, die dauerhaft mit Einschränkungen leben — Sehbehinderungen, eingeschränkte Motorik, Gehörlosigkeit, kognitive Einschränkungen.

2 Und situativ: jeder von uns in bestimmten Momenten. Grelles Sonnenlicht, eine Hand voll Kaffee, Ton aus in der Bahn, schlechte Verbindung unterwegs.

3 Dieselbe Barriere, unterschiedliche Ursachen. Schlechter Farbkontrast schließt jemanden mit Farbsehschwäche aus — und jeden, der im Freien auf sein Handy schaut. Keine Untertitel? Schlecht für Gehörlose, und für jeden, der gerade den Ton nicht aufdrehen kann.

Das ist der eigentliche Grund, warum Accessibility kein Randthema ist: Wir bauen für uns selbst.



Accessibility basiert auf vier Prinzipien, zusammengefasst als POUR. Das ist der Standard der WCAG.

1 Erstens: Perceivable, sprich Wahrnehmbar. Informationen müssen für Menschen mit allen Sinnen zugänglich sein. Alt-Text für Bilder, damit Screen Reader sie beschreiben können. Guter Kontrast, damit man sie auch im Freien liest. Untertitel für Videos, damit Gehörlose folgen können.

2 Zweitens: Operable, oder Bedienbar. Jeder muss die Anwendung mit den Eingabemitteln steuern können, die er hat. Keyboard-Navigation für Menschen, die die Maus nicht benutzen. Buttons müssen außerdem groß genug sein, damit man sie auf Touchscreens sicher treffen kann. Der Fokus muss sichtbar sein, damit man weiß, wo man ist. Und keine versteckten Zeitlimits — nicht alle lesen im selben Tempo.

3 Drittens: Understandable, also Verständlich. Die Struktur und Logik müssen klar sein. Semantisches HTML, das die Bedeutung ausdrückt. Labels für Inputs, damit du weißt, was hineingehört. Aussagekräftige Fehlermeldungen statt kryptischer Codes.

4 Viertens: Robust. Es muss funktionieren, egal welchen Browser oder welche Assistive-Technology jemand nutzt. Valid HTML, damit alles richtig geparkt wird. ARIA korrekt eingesetzt, nicht nur dekorativ. Responsive, damit es auf jedem Screen funktioniert.

Alle vier Prinzipien greifen ineinander. Wer einen verbessert, erleichtert die Nutzung für viele.

Mini Accessibility-Audit

BEISPIEL

Zurücksetzen


```

<form id="search-form">
  <input placeholder="Suche"
    name="search-term" />

  <div>
    <button>
      
    </button>
    <div>Zurücksetzen</div>
  </div>
</form>

```

Charlie Harding · QAware GmbH 45

Wir machen jetzt keinen Deep-Dive, sondern einen Mini-Audit wie im echten Review.

Links ein typischer Ausschnitt: Suchfeld und Aktionen. Auf den ersten Blick „funktioniert es“.

Rechts sieht ihr das zugrunde liegende HTML.

Erstens: Input ohne sichtbares Label.

Mini Accessibility-Audit

BEISPIEL

Suche

Die Tab-Reihenfolge stimmt mit der visuellen Reihenfolge überein.

```
<form id="search-form">
  <label>Suche
    <input name="search-term" />
  </label>

  <div>
    <button>Zurücksetzen</button>
    <button aria-label="Suchen">
      
    </button>
  </div>
</form>
```

Charlie Harding QAware GmbH 46

1 Ein Platzhalter reicht nicht, weil er verschwindet, sobald man etwas eingibt.

2 Mit einem Label sieht es besser aus.

Allerdings ist es nicht leicht zu lesen: Der Kontrast ist zu gering.

3 Und sieht: Das Label ist jetzt deutlich sichtbar – viel besser lesbar.

Die Buttons sind auch recht klein.

4 Viel besser: Jetzt sind die Touch-Targets groß genug und mobil zuverlässig zu treffen.

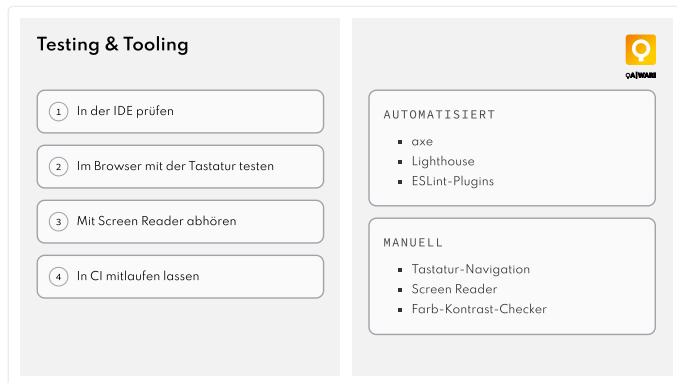
Diese interaktiven Elemente brauchen außerdem echte Semantik: also Button oder Link statt klickbarem div, damit sie für Screen-Reader und Tastaturen besser bedienbar sind.

5 Und wenn ein Button nur ein Icon hat, wird er bei einer schlechten Internetverbindung oder beim Request-Fehler unbrauchbar. 6 Wir brauchen einen zugänglichen Namen, bspw. per **aria-label** oder **alt**.

7 Der Tastatur-Fokus muss sichtbar sein. Mit nativen Buttons kriegt man das automatisch, aber man muss aufpassen, es nicht wieder zu überschreiben.

Die Reihenfolge muss auch passen. Manchmal kann es in CSS einfacher sein, die Reihenfolge umzukehren, um Buttons zu positionieren, aber das verschlechtert die Bedienbarkeit für Tastatur- und Screen-Reader-Nutzer.

8 Diese Reihenfolge sieht jetzt besser aus.



Nachdem wir die häufigsten Fehler gesehen haben, geht es darum, wie wir sie früh entdecken.

➤ 1 Automatisierte Checks wie axe, Lighthouse und ESLint-Plugins sind schnell und gut für klare Regeln. Sie finden sofort, was offensichtlich kaputt ist.

➤ 2 Dann prüfen wir manuell: Tastatur, ➤ 3 Screen Reader und Kontrast zeigen, ob sich die Bedienung wirklich gut anfühlt.

➤ 4 Und schließlich gehört das in den normalen Workflow: in der IDE, im Browser und in CI.

So wird Accessibility ein normaler Teil der Entwicklung und nicht nur ein letzter Haken am Ende.



Zum Abschluss: Accessibility ist kein eigener Arbeitsschritt am Ende, sondern tägliche Praxis.

➤ 1 Im Review prüfen wir nicht nur Logik und Stil, sondern auch Themen wie Labels, Fokuszustände und Tastatur-Reihenfolge.

➤ 2 In gemeinsamen UI-Komponenten zählt sich das doppelt aus: Wenn Basisbausteine barrierearm gebaut sind, profitieren alle Features automatisch.

➤ 3 Im Test kombinieren wir schnelle automatisierte Checks mit gezielten manuellen Tests, z. B. mit Tastatur und Screen Reader.

➤ 4 Und im Team gehört das in die Fertig-Checkliste: Was nicht zugänglich ist, ist noch nicht fertig. So wird Accessibility kein Zufall, sondern Standard.

➤ 5 Genau so wird aus Accessibility verlässliche Produktqualität.

Und genau da liegt der nächste Schritt: Wenn wir diese Qualitätsentscheidungen nicht in jedem Feature neu treffen wollen, brauchen wir gemeinsame Bausteine und Regeln, die sie standardmäßig mitbringen.



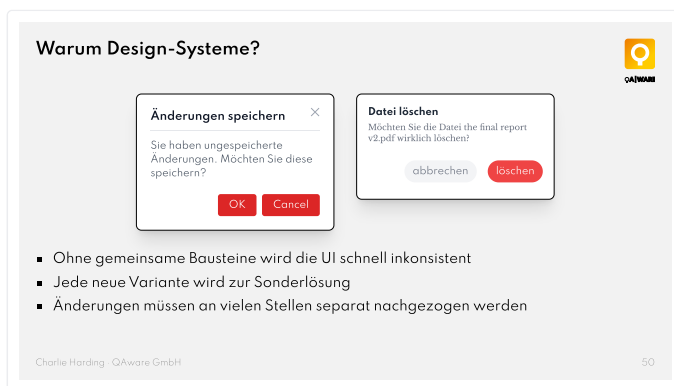
Bis jetzt ging es viel um Qualität im Alltag: gute Regeln im Review, im Test und im Team.

Jetzt kommt der nächste Schritt: Wie machen wir diese Qualität skalierbar, sodass nicht jede Person dieselben Probleme immer wieder neu löst?

Genau dafür sind Design-Systeme da: gemeinsame Bausteine, gemeinsame Regeln, gemeinsame Sprache.

Es geht also nicht nur um „schöne UI“, sondern um Konsistenz, Geschwindigkeit und weniger Reibung im Team.

Kurz: Von Einzellösungen zu wiederverwendbaren Patterns.



Wir gehen nochmal zurück zu dem Beispiel von vorhin.

Am Anfang haben wir darauf geschaut und gesagt: Das sieht inkonsistent aus. Zwei Dialoge, zwei unterschiedliche Stile, zwei unterschiedliche Regeln.

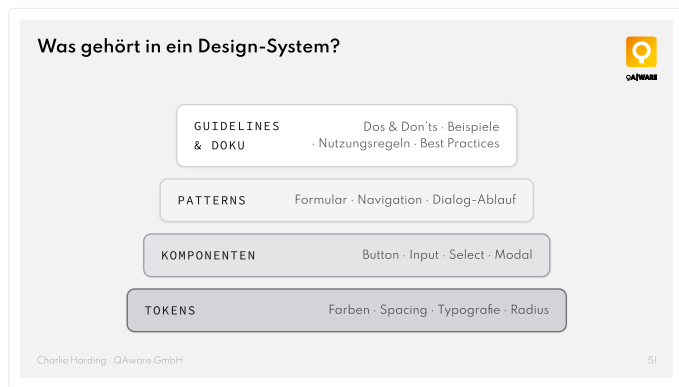
Jetzt schauen wir mit einer anderen Frage darauf: Warum passiert das überhaupt?

1 Die einfache Antwort ist: Es gibt keine gemeinsamen Bausteine. Jeder baut seinen eigenen Dialog, seinen eigenen Button, seine eigene Variante.

2 Und damit wird jede kleine Anforderung sofort zur Sonderlösung. Noch ein anderer Dialog. Noch ein anderer Button. Noch eine Ausnahme.

3 Das Problem ist also nicht nur optische Inkonsistenz. Änderungen müssen dann an vielen Stellen separat nachgezogen werden — und das wird oft einfach vergessen oder nicht einheitlich gemacht.

Genau an diesem Punkt werden Design-Systeme interessant: Sie schaffen gemeinsame Bausteine und gemeinsame Regeln.



Wenn man nur von Komponenten spricht, greift das bei Design-Systemen zu kurz.

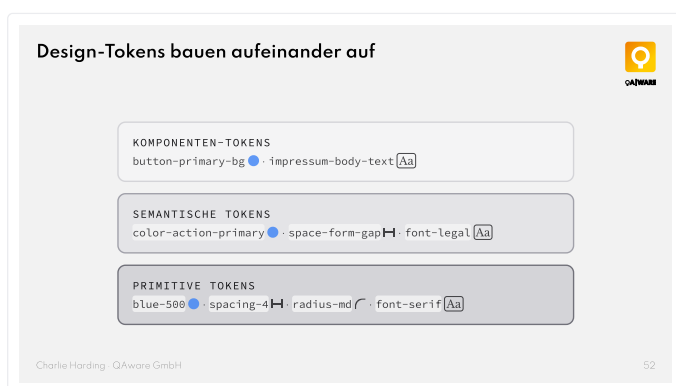
1 Ganz unten liegen die Tokens: also Farben, Abstände, Typografie oder Radius. Das sind die kleinsten wiederverwendbaren Designentscheidungen.

2 Darauf bauen die Komponenten auf: Button, Input, Select, Modal. Das ist die Ebene, an die viele zuerst denken.

3 Eine Ebene darüber liegen Patterns. Also wiederkehrende Kombinationen von Komponenten, zum Beispiel Formulare, Navigation oder Dialog-Abläufe.

4 Und ganz oben braucht es Guidelines und Dokumentation: Wann benutze ich was? Welche Regeln gelten? Was sind gute und schlechte Beispiele?

Genau dadurch wird aus einer Sammlung von Bausteinen ein Design-System.



Ein häufiger Fehler ist: Tokens als eine flache Liste zu sehen.

1 Unten starten wir mit primitiven Tokens: reine Designwerte wie Farben, Abstände, Radius und Schriftarten.

2 Darauf bauen semantische Tokens auf. Die geben Bedeutung statt Rohwerten, zum Beispiel für primäre Aktionen, Formular-Abstände oder typografische Rollen.

3 Und ganz oben liegen Komponenten-Tokens: konkrete Ableitungen pro Baustein wie Button-Hintergründe oder Textstile in einzelnen Komponenten.

Dadurch baut alles aufeinander auf. Wir ändern unten einmal sinnvoll — und oben passt sich das Verhalten konsistent an.

53

Beispiel: Konsistente Dialoge

Änderungen speichern ✕
 Sie haben ungespeicherte Änderungen. Möchten Sie diese speichern?
 OK Cancel

Datei löschen
 Möchten Sie die Datei the final report v2.pdf wirklich löschen?
 abbrechen löschen

Welche **gewollten** Unterschiede sieht ihr?

Charlie Harding · QAware GmbH 53

Wir nutzen nochmal dasselbe Beispiel wie am Anfang.

Diesmal ist die Aufgabe aber eine andere: Nicht einfach nur Unterschiede finden, sondern die gewünschten Unterschiede erkennen. Welche seht ihr?

54

Beispiel: Konsistente Dialoge

Änderungen speichern ✕
 Sie haben ungespeicherte Änderungen. Möchten Sie diese speichern?
 OK Cancel

Datei löschen
 Möchten Sie die Datei the final report v2.pdf wirklich löschen?
 abbrechen löschen

Eigenschaft	Links	Rechts
Titeltext	„Änderungen speichern“	„Datei löschen“
Body-Text	„Sie haben ungespeicherte Änderungen. Möchten Sie diese speichern?“	„Möchten Sie die Datei the final report v2.pdf wirklich löschen?“
Primary-Action	primary	destructive
Primary-Action-Label	Speichern	Löschen

Die Titeltexte sollen sich natürlich unterscheiden.

1 Auch die Body-Texte sind unterschiedlich, weil sie unterschiedliche Informationen vermitteln. Hier sind sie Strings, aber wahrscheinlich wollen wir dynamischen Inhalt unterstützen, damit beispielsweise der Dateiname oder die Anzahl der ungespeicherten Änderungen automatisch eingefügt werden kann, am besten schön formatiert.

2 Auf der rechten Seite wird etwas gelöscht, also kennzeichnen wir die destruktive Aktion auch visuell, z. B. mit roter Farbe. Diese Werte entsprechen den Tokens, die das Design-System bereitstellt.

3 Und schließlich ist das Label der Primary Action unterschiedlich: OK vs. Löschen. Wir können das tatsächlich verbessern: Speichern ist eindeutiger als OK.

Der Unterschied beim Abbrechen-Label müssen wir nicht berücksichtigen: Vorher war es eher verwirrend, dass der linke Dialog den englischen Text „Cancel“ hatte.

Genau diese Inputs brauchen wir erstmal, um eine Komponente in unserem Design-System zu bauen. Aber danach können wir sie überall wiederverwenden — mit denselben Regeln, derselben Accessibility, und derselben Konsistenz.

55

Beispiel: Konsistente Dialoge

Änderungen speichern
 Sie haben ungespeicherte Änderungen. Möchten Sie diese speichern?
 Abbrechen Speichern

Datei löschen
 Möchten Sie die Datei the final report v2.pdf wirklich löschen?
 Abbrechen Löschen

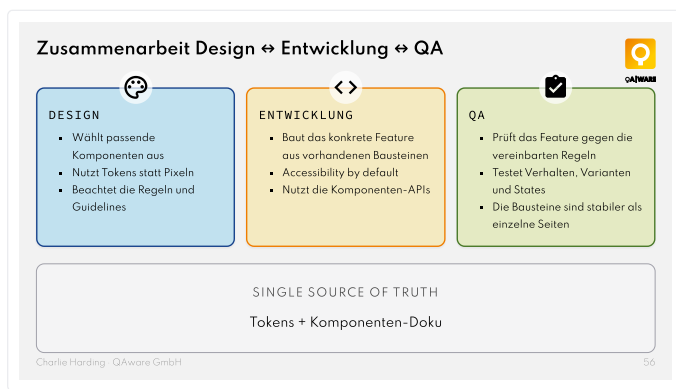
```

<ui-dialog
  title="Änderungen speichern"
  primary-button-text="Speichern"
>
  Sie haben ungespeicherte Änderungen.
  Möchten Sie diese speichern?
</ui-dialog>

<ui-dialog
  title="Datei löschen"
  primary-button-variant="destructive"
  primary-button-text="Löschen"
>
  Möchten Sie die Datei
  <code>the final report v2.pdf</code>
  wirklich löschen?
</ui-dialog>
  
```

So würde das dann im Code aussehen,

1 und jetzt sind die Dialoge komplett konsistent. Jeder zukünftige Dialog wird auch genauso aussehen, ohne dass man überhaupt HTML oder CSS schreiben muss.



Sobald die Bausteine des Design-Systems stehen, wird die Zusammenarbeit zwischen Design, Entwicklung und QA viel einfacher.

1 Design wählt passende Bausteine aus, definiert das gewünschte UI-Pattern und liefert Entwürfe anhand von Tokens und Komponenten statt Pixelwerte.

2 Entwicklung baut daraus das konkrete Feature zusammen. Weil Design schon kennt, welche Konfig-Optionen die Komponenten anbieten, sind die Entwürfe viel einfacher umzusetzen. Und Accessibility ist direkt eingebaut, weil die Bausteine das schon berücksichtigen.

3 QA testet das fertige Feature: Stimmen Verhalten, Varianten und States im Zusammenspiel der Bausteine?

4 Damit das funktioniert, brauchen alle dieselbe Quelle: Tokens, Komponentendoku und Design-System-Guidelines.

Das führt zu besseren Übergaben, weniger Missverständnisse und schnelleren Releases.

Bestehende Systeme zuerst?



Material Design

- Von Google
- Eine UI, die sehr Android-nah wirkt
- Angular, aber auch Ports für React, Vue, etc.
- <https://material.angular.dev/>

Primer

- Von GitHub
- Fokus auf Konsistenz und Accessibility
- Nur React, aber auch CSS-Only-Variante
- <https://primer.style/>

- Startet mit einem bestehenden System, wenn es gut zum Produkt passt
- Baut eigene Bausteine nur dort, wo ihr euch bewusst unterscheiden wollt

Bevor man selbst ein Design-System baut, lohnt sich oft ein Blick auf bestehende Systeme.

Es gibt natürlich recht viele, hier sind nur ein paar Beispiele:

1 Material Design kommt von Google und ist sehr vollständig, wirkt aber je nach Produkt schnell sehr Android-nah. Es gibt eine offizielle Angular-Implementierung, aber auch Ports für React, Vue und andere Frameworks.

2 Primer ist Githubs eigenes System, entwickelt mit Fokus auf Konsistenz und Accessibility. Es ist vor allem React-fokussiert, aber es gibt auch eine CSS-Only-Variante, falls ihr etwas anderes nutzen möchtet.

Wenn man etwas schnell bauen will, ist es oft sinnvoll, mit einem bestehenden System zu starten. Das spart Zeit und liefert direkt viele Bausteine.

3 Wichtig ist die Strategie: Übernehmen, was passt.

4 Und nur dort eigene Bausteine bauen, wo man sich wirklich unterscheiden möchte.

Einführung im Alltag



- Klein starten: 3-5 Kernkomponenten mit klaren Standards
- Gemeinsame Arbeitsweise festlegen: Wer pflegt was?
- Schrittweise Migration: neue Features zuerst
- Altbestand nur anfassen, wenn ihr dort sowieso arbeitet

Qualität wird skalierbar, wenn gute Bausteine zur Standardlösung werden.

Zum Abschluss geht es darum, wie man das im Alltag einführt.

1 Nicht mit 30 Komponenten starten, sondern mit wenigen Kernbausteinen.

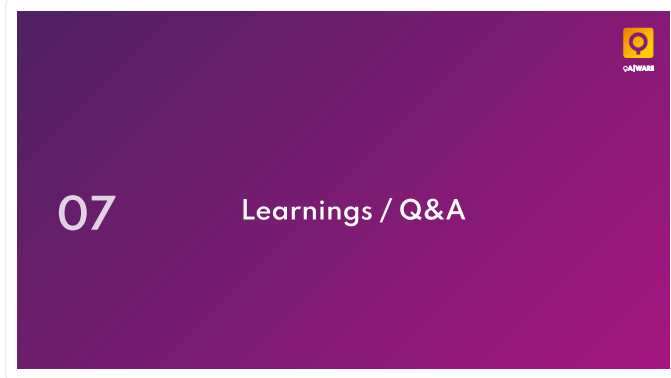
2 Dann klären, wer die Bausteine pflegt und wie Änderungen abgestimmt werden.

3 Bei der Migration pragmatisch sein: neue Features zuerst.

4 Alten Code nur anfassen, wenn ihr dort ohnehin arbeitet.

5 So wird Qualität Schritt für Schritt skalierbar, statt ein Big-Bang-Projekt zu werden.

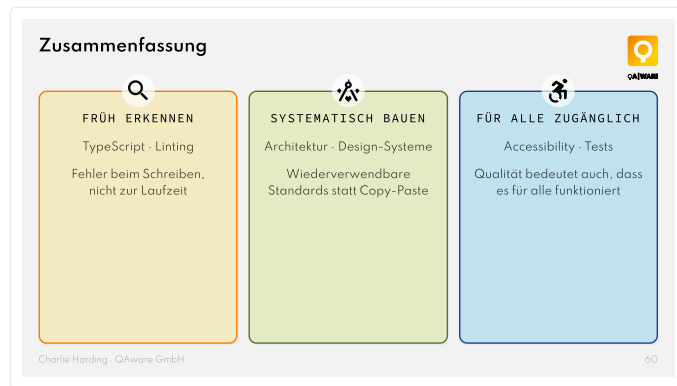
59



Zum Abschluss ziehen wir die wichtigsten Punkte nochmal zusammen.

Danach stelle ich euch die heutige Übung vor — und dann ist noch Zeit für eure Fragen.

60



Was haben wir heute gelernt?

1 Erstens: Probleme früh erkennen. Mit TypeScript und Linting fassen wir Fehler beim Schreiben, nicht erst wenn der Nutzer sie trifft.

2 Zweitens: Systematisch bauen. Architektur und Design-Systeme schaffen wiederverwendbare Standards, statt dass jedes Team das Rad neu erfindet.

3 Drittens: Für alle zugänglich. Qualität bedeutet auch, dass die Lösung für alle Menschen funktioniert — nicht nur für die ideale Bedingung.

Das ist die Kernidee: Qualität ist nicht eines, es ist ein System.

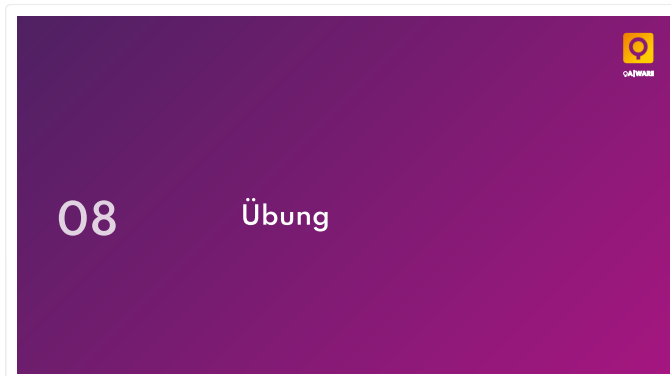
61



Bevor wir zur Übung kommen, haben wir jetzt Zeit für Fragen.

Was wollt ihr noch wissen? Was war unklar?

62



No notes for this slide

63

Aufgabe

- Startet mit einer kurzen Erkundung: App ausprobieren, Codebasis durchklicken, Probleme notieren
- Arbeitet in 2er-Teams am Starter-Repo
- Ziel: Das Frontend Schritt für Schritt verbessern
- Nutzt dafür TypeScript, Linting, Accessibility und konsistente Komponenten

Nicht alles perfekt machen - lieber die wichtigsten Probleme sauber lösen.



Charlie Harding · QAware GmbH
63

Jetzt geht es in die Praxis.

Wichtig als erster Schritt: Schaut euch die App und den Code erst einmal bewusst an, probiert typische Flows aus und notiert euch gefundene Probleme.

Ihr arbeitet alleine oder in 2er-Teams am Starter-Repo und verbessert es Schritt für Schritt.

Wichtig: Nicht alles perfekt machen wollen. Priorisiert zuerst kritische Probleme, dann wichtige, dann Nice-to-have.

Fokus für die Übung:

- TypeScript: klare Typen statt **any**
- Linting: Tooling aufsetzen, Warnungen systematisch aufräumen
- Accessibility: Labels, Fokus, Tastaturbedienung, Kontrast
- Konsistenz: gleiche Muster und Komponenten einheitlich nutzen
- Design-System: Wiederverwendbare Komponenten extrahieren

64

Ressourcen

Links

- Slides <https://c-harding.github.io/frontend-quality-talk/>
- Handout <https://c-harding.github.io/frontend-quality-talk/main/notes-export.pdf>
- Exercise Repository <https://github.com/c-harding/frontend-quality/>

Checkliste

- Erst analysieren: Probleme sammeln und priorisieren
- TypeScript: Typen schärfen statt any
- Linting aktivieren und Fehler beheben
- Accessibility: Labels, Fokus, Tastaturbedienung, Kontrast
- Konsistenz: gemeinsame Komponenten statt Duplicate-Code
- Design-System-Denken: wiederverwendbare Bausteine schneiden

Alle Setup-Schritte und Hinweise findet ihr in der README



Charlie Harding · QAware GmbH

Hier sind die wichtigsten Ressourcen für die Übung.

Links: Erstens die Slides, zweitens das Exercise-Repository.

Der Fokus für die Bearbeitung steht rechts als Checkliste. Wenn ihr priorisieren müsst: erst TypeScript und Tooling aktivieren, dann Accessibility und Konsistenz.

Wichtig: Alles Organisatorische steht in der README im Exercise-Repo - Setup, Ablauf und Definition of Done.

65

qaware.de


Charlie Harding
charlie.harding@qaware.de

 [c-harding/frontend-quality](https://github.com/c-harding/frontend-quality)
 [c-harding/frontend-quality-talk](https://github.com/c-harding/frontend-quality-talk)

No notes for this slide